

Reviewer Pack — Noncommutative Fourier Coefficient Inversion in Greater-Than...

Entry 5f49c0d34464 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 00:29:23 UTC

Noncommutative Fourier Coefficient Inversion in Greater-Than Communication Complexity

Entry ID: 5f49c0d34464 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 00:29:23 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Harmonic Analysis **Field B** (complexity object): Communication Complexity of the Greater-Than Function

Statement:

For a Boolean function f on n variables, the noncommutative Fourier coefficients at irreducible representations λ are inversely proportional to the communication complexity of the Greater-Than function on n bits. Specifically, $|F_\lambda(f)| \geq \Omega(\log n) / CC_GT(n)$, where $CC_GT(n)$ is the communication complexity of the Greater-Than function.

Rationale (proposer's reasoning):

Noncommutative Fourier analysis reveals structural properties of Boolean functions that correlate with communication complexity. By mapping Boolean functions to noncommutative polynomials and analyzing their coefficients, we can derive bounds on communication complexity for specific problems like Greater-Than.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5fa6108fac14168f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 5 + (seed % 40) // 2 # Sweep n through {5, 10, 15, 20, 30, 40}
    if n == 1:
        return {
            "metric_name": "CC_GT(n)",
            "metric_value": 1,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "single_output_stub"
        }

    # Generate a random Boolean function f on n variables
    f = [random.choice([0, 1]) for _ in range(2**n)]

    # Compute the communication complexity of the Greater-Than function
    CC_GT_n = math.log(n, 2)

    # Compute the noncommutative Fourier coefficient at irreducible representation  $\lambda$ 
     $\lambda$  = n // 2 # Example: using a simple irreducible representation for demonstration
    F_lambda_f = sum(f[i] * math.exp(-2j * math.pi * i *  $\lambda$ ) for i in range(2**n)) / (2**n)

    # Compute the magnitude of the Fourier coefficient
    metric_value = abs(F_lambda_f)

    # Check if the conjecture holds
    conjecture_holds = metric_value >= CC_GT_n / math.log(n, 2)
    counterexample = "" if conjecture_holds else f"CC_GT({n})={CC_GT_n}, |F_lambda(f)|={metric_value}"

    return {
        "metric_name": "CC_GT(n)",
        "metric_value": metric_value,
        "instances_tested": 1,
```

```

        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c33fdfca.py", line 62, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c33fdfca.py", line 39, in run_trial
    F_lambda = sum(f[i] * math.exp(-2j * math.pi * i * lambda) for i in range(2**n)) / (2**n)
                ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c33fdfca.py", line 39, in <genexpr>
    F_lambda = sum(f[i] * math.exp(-2j * math.pi * i * lambda) for i in range(2**n)) / (2**n)
                    ~~~~~^
TypeError: must be real number, not complex

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to complex number type error, preventing data collection. Support condition not met with incomplete results. | next: Fix type handling for complex numbers in Fourier coefficient calculation and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	141006
2	propose	ollama_remote	qwen3:8b	0	0	154413
3	preregistration	ollama_remote	qwen3:8b	0	0	31758
4	novelty	ollama_remote	qwen3:8b	0	0	27267
5	novelty	ollama_remote	qwen3:8b	0	0	19858
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10439
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7578
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7493
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8257
10	judge	ollama_remote	qwen3:8b	0	0	22777

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 430846 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5f49c0d34464.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5f49c0d34464.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5f49c0d34464.tar.gz` (if generated)